MIDTERM REPORT: REIMPLEMENTING PAGEDATTENTION IN MINITORCH

NingRui Yang [1] **Yixiang Dai** [1]

ABSTRACT

We present a from-scratch reimplementation of PagedAttention within the MiniTorch educational deep learning framework. PagedAttention (Kwon et al., 2023) eliminates KV cache fragmentation during LLM inference by managing cache memory in non-contiguous blocks via a block table, analogous to virtual memory paging. Our implementation spans three layers: (1) a Python-level block memory manager with reference-counted allocation and fragmentation tracking, (2) a custom CUDA kernel (`paged_attention_v1_kernel`) that computes scaled dot-product attention directly over non-contiguous cache blocks using warp-level reductions, and (3) a transformer integration that supports both prefill and autoregressive decode phases. As of the midterm checkpoint, all three components are functionally complete and pass 15 correctness tests, including end-to-end tests that validate the CUDA kernel output against a Python reference implementation within 10^{-4} tolerance. We discuss the system-level challenges encountered—notably, a CUDA runtime/driver API context conflict between PyTorch, Numba, and our custom kernel—and outline the remaining work on benchmarking and optimization.

1 INTRODUCTION

Large Language Model (LLM) inference is bottlenecked by the Key-Value (KV) cache, which stores the projected key and value vectors for all previously generated tokens and grows linearly with context length. Production serving systems typically allocate KV cache memory in large, contiguous blocks—one per request at maximum sequence length—leading to three compounding problems:

1. **Internal fragmentation.** Pre-allocated blocks waste memory on slots that are never filled because generation terminates early.

2. **External fragmentation.** As requests arrive and depart, freed blocks leave non-contiguous holes that cannot be coalesced.

3. **Reduced concurrency.** Together, these effects artificially limit the batch size that fits in GPU memory, under-utilizing both memory and compute.

PagedAttention (Kwon et al., 2023) addresses this by borrowing the paging abstraction from operating systems: the KV cache is divided into small, fixed-size *blocks* (pages) that need not be contiguous in physical GPU memory. A per-sequence *block table* maps logical token positions to physical block identifiers at runtime, enabling near-zero-waste memory utilization and on-demand block allocation.

Motivation. While PagedAttention is deployed in production systems such as vLLM (Kwon et al., 2023), Hugging Face TGI (Hugging Face, 2023), and TensorRT-LLM (NVIDIA, 2024), existing implementations are tightly coupled to large, heavily optimized codebases that obscure the core algorithmic ideas. By reimplementing PagedAttention within MiniTorch—a minimal educational framework from CMU 11-868—we aim to provide a self-contained, readable implementation that exposes the system-level design decisions without the complexity of a production system.

Contributions. At the midterm checkpoint, our contributions are:

1. A block-based KV cache memory manager with allocation, deallocation, reference counting, and fragmentation metrics.

2. A custom CUDA kernel that performs scaled dot-product attention directly over non-contiguous memory blocks in three passes (QK scoring, stable softmax, value accumulation) with warp-level reductions.

3. Integration of the above into a MiniTorch transformer model supporting both prompt prefill and single-token decode.

4. A comprehensive test suite (15 tests) validating correctness from individual block operations through end-to-

[1]ECE, Carnegie Mellon University, Pittsburgh, PA, US. **AUTHORERR: Missing \mlsyscorrespondingauthor.**

Proceedings of the 5th MLSys Conference, Santa Clara, CA, USA, 2024. Copyright 2024 by the author(s).

end autoregressive generation.

2 Related Work and Background

PagedAttention and vLLM. Kwon et al. (Kwon et al., 2023) introduced PagedAttention as the core memory management mechanism in the vLLM serving engine. Their approach partitions the KV cache into fixed-size blocks (typically 16 tokens) and introduces a block table for logical-to-physical mapping. The original CUDA kernels are highly optimized with template specialization for head dimensions, multi-level tiling, and warp-level partitioning. Our implementation follows the same algorithmic structure (the "V1 kernel" in vLLM terminology) but prioritizes clarity over peak performance: we use a single kernel variant with dynamic shared memory rather than compile-time template parameters.

FlashAttention. FlashAttention (Dao et al., 2022) and FlashAttention-2 (Dao, 2024) optimize attention computation through IO-aware tiling and kernel fusion, reducing HBM reads by keeping partial softmax statistics on-chip. However, FlashAttention assumes contiguous KV storage and does not address the memory *management* problem of dynamic allocation during autoregressive serving. FlashInfer (Ye et al., 2025) extends flash-style kernels to support both ragged and paged KV cache layouts, bridging the gap between computation efficiency and memory flexibility. Our work is closest to vLLM's V1 kernel but embedded in an educational framework.

Operating System Analogies. The paging metaphor in PagedAttention draws directly from OS virtual memory management (Silberschatz et al., 2018). A block table serves the role of a page table, mapping logical token indices to physical block IDs. Our block manager implements a simplified version of this with FIFO free-list allocation, reference counting (analogous to shared pages), and fragmentation tracking similar to an OS memory metrics module.

MiniTorch. MiniTorch is a pedagogical deep learning library used in CMU 11-868 (Rush, 2020). It provides tensor operations, automatic differentiation, and CUDA backends via Numba JIT (driver API) and custom `.cu` kernels loaded through ctypes (runtime API). We build on the hw3/hw4 codebase, extending it with paged attention, transformer generation support, and system-level runtime fixes.

3 Methodology

Our implementation addresses three research questions:

RQ1 Can a simple block-based allocator reproduce the near-zero fragmentation benefits of PagedAttention?

```
Sequence 0 (7 tokens, block_size=4)

Block Table: [2, 5]
Logical:  [tok0..tok3] [tok4..tok6, _]
Physical: block_2       block_5

key_cache[2, 0:4, :, :] = K for tok0..3
key_cache[5, 0:3, :, :] = K for tok4..6
```

Figure 1. Logical-to-physical mapping via the block table. Sequence 0 has 7 tokens distributed across physical blocks 2 and 5. Block 5 has 1 empty slot (internal fragmentation).

RQ2 Can a single, non-templated CUDA kernel correctly compute attention over non-contiguous cache blocks?

RQ3 What system-level integration challenges arise when combining custom CUDA runtime API code with MiniTorch's existing Numba + PyCUDA backends?

3.1 Block Memory Manager

The `BlockManager` (Figure 1) manages a pool of N physical blocks, each holding B token slots (default $B = 16$). Key data structures include:

- **Free list** (`free_block_ids`): a FIFO queue of available block IDs.
- **Block metadata** (`KVBlock`): per-block tracking of fill count, reference count, and capacity.
- **Block table** (`BlockTable`): a per-sequence mapping from logical block indices to physical block IDs.
- **Global KV cache**: two NumPy arrays of shape (N, B, H, D) where H is the number of attention heads and D is the head dimension.

The allocation protocol works as follows: `allocate_blocks_for_sequence` computes $\lceil T/B \rceil$ blocks for a T-token prompt, pops block IDs from the free list, creates a `BlockTable`, and distributes fill counts. `append_token_to_sequence` checks whether the tail block is full; if so, it allocates a new block before incrementing the fill count. `free_sequence` decrements reference counts and returns zero-reference blocks to the free list, zeroing their cache slices.

We track two fragmentation metrics: *internal fragmentation* measures wasted slots within allocated blocks ($1 -$ used$/$capacity), and *external fragmentation* measures the fraction of free blocks trapped between the minimum and maximum used block IDs.

3.2 CUDA Kernel

The `paged_attention_v1_kernel` computes attention for one (sequence, head) pair per thread block, with grid dimensions $(\text{batch_size}, n_\text{head})$. The thread count is set to $\max(32, \min(\text{head_dim}, 1024))$ to ensure at least one full warp for warp-level primitives.

The kernel uses dynamic shared memory partitioned as:

$$\underbrace{\text{logits}[\text{ctx_len}]}_{\text{Pass 1–2}} \parallel \underbrace{\text{out_accum}[D]}_{\text{Pass 3}} \parallel \underbrace{\text{warp_scratch}[W]}_{\text{reductions}}$$

where $W = \lceil \text{threads}/32 \rceil$ is the number of warps.

Pass 1: QK Scoring. Each thread iterates over the context tokens it owns (stride = num_threads). For each token, it maps the logical index to a physical cache location via the block table:

$$\begin{aligned} \text{logical_block} &= \lfloor t/B \rfloor \\ \text{slot} &= t \bmod B \\ \text{physical_block} &= \text{block_table}[\text{logical_block}] \end{aligned}$$

It then computes the full dot product $\mathbf{q} \cdot \mathbf{k}_t / \sqrt{D}$ and stores the result in shared memory.

Stable Softmax. A global maximum is found via a two-level reduction: intra-warp reduction with `__shfl_xor_sync` (`warp_reduce_max`), followed by inter-warp reduction through shared memory. Each thread then computes $\exp(\text{logit}_t - \max)$ and the sum is reduced using the same warp-then-block pattern. Finally, each logit is normalized in-place to produce attention weights.

Pass 3: Value Accumulation. Each thread accumulates the weighted sum $\sum_t w_t \cdot v_t[d]$ for the head dimensions it owns, again performing block-table lookups on every value access. The result is written to global memory.

Host Launcher. The extern "C" launcher manages GPU memory explicitly: it allocates device buffers via `cudaMalloc`, copies inputs host-to-device, launches the kernel, synchronizes, copies the output device-to-host, and frees all device memory. The `.so` is compiled with `--cudart shared` to share the process-wide CUDA runtime library, avoiding conflicts with Numba's driver API context (see Section 5.1).

3.3 Transformer Integration

The transformer model consists of `PagedDecoderLM` → `PagedTransformerLayer` → `PagedMultiHeadAttention`, following a Pre-LN architecture (Xiong et al., 2020).

Table 1. Test suite results. All tests pass with the CUDA kernel validated against the Python reference within 10^{-4} tolerance.

Test Class	What is tested	Pass
`StandardAttention`	Identity, causal mask	2/2
`PagedVsStandard`	Single/multi block, batch, block size sweep	4/4
`PagedMultiHeadAttn`	Prefill cache, decode	2/2
`PagedTransformerLayer`	Shape, residual conn.	3/3
`PagedDecoderLM`	Generate, free seqs	2/2
`PagedAttnKernel`	CUDA single & batch	2/2
Total		**15/15**

Prefill Phase. Given a prompt of length T, the model projects Q, K, V via linear layers, applies standard causal masked attention over the full prompt (contiguous computation), and populates the KV cache through the block manager.

Decode Phase. For each new token, Q, K, V are projected for the single-token input. The new KV pair is appended to the cache via `write_token_kv()`, potentially allocating a new block. Attention is then computed over all cached tokens using `paged_attention_ref()`, which gathers non-contiguous blocks through the block table for each sequence.

Generation Loop. The `generate()` method orchestrates the full autoregressive loop: prefill → sample → decode × (`max_new_tokens` − 1), with temperature-controlled sampling (softmax + multinomial or argmax) and cleanup via `free_sequence()`.

4 Experiments

4.1 Correctness Validation

We validate correctness through a comprehensive test suite organized into six test classes covering the full stack from individual block operations to end-to-end generation. Table 1 summarizes the results: all 15 tests pass.

Kernel Correctness. The two CUDA kernel tests construct random queries, keys, and values, populate a paged cache via `build_paged_cache()`, and compare the kernel output against `paged_attention_ref()` using `np.testing.assert_allclose` with atol = rtol = 10^{-4}. The single-sequence test uses 2 heads, head dimension 4, block size 4, and context length 7 (spanning 2 blocks). The batched test uses 3 sequences with context lengths $[3, 7, 5]$ and padded block tables, verifying correct handling of variable-length inputs.

Paged vs. Standard Attention. Four tests compare `paged_attention_ref()` against

`standard_attention()` to ensure the paging indirection preserves correctness. A block-size sweep over $B \in \{1, 4, 8, 16, 32\}$ demonstrates robustness to this hyperparameter, with all configurations matching within 10^{-5}.

End-to-End Generation. The `TestPagedDecoderLM` tests instantiate a small model (vocab 50, embedding 32, 2 heads, 2 layers) and run `generate()` for 5 new tokens. Tests verify correct output shape and that `free_sequence()` returns all blocks to the free pool after generation completes.

4.2 Benchmarking (Planned)

Quantitative performance benchmarks are planned for the final report and will include:

- **Throughput**: tokens/second for prefill and decode, comparing paged vs. contiguous attention.
- **Memory utilization**: peak KV cache bytes and utilization ratio under varying batch sizes and sequence lengths.
- **Fragmentation**: internal and external fragmentation under synthetic workloads with dynamic request arrivals and departures.
- **Max batch size**: largest batch achievable before OOM, comparing paged allocation against pre-allocated contiguous KV.

5 Analysis

5.1 CUDA Runtime vs. Driver API Context Conflict

A significant engineering challenge arose from the interaction between three CUDA subsystems used concurrently in MiniTorch:

1. **PyTorch** – uses the CUDA *runtime* API (`cudaMalloc`, `cudaMemcpy`).
2. **Numba** – uses the CUDA *driver* API (`cuCtxCreate`) for JIT-compiled kernels in `cuda_ops.py`.
3. **Our kernel** – compiled as a shared library (`.so`) and invoked via ctypes, using the CUDA *runtime* API in its host launcher.

When MiniTorch's `__init__.py` imported `cuda_ops` (which triggers `numba.cuda`), the Numba driver API created a CUDA context *before* the runtime API was initialized. This caused all subsequent runtime API calls—`cudaSetDevice`, `cudaMalloc`, `cudaMemcpy`—to fail with "no CUDA-capable device is detected," producing all-zero output from the kernel.

The root cause was two-fold:

1. The `paged_attention.so` was compiled with *static* CUDA runtime linking (the `nvcc` default), giving it a private copy of `libcudart` that could not interoperate with the existing driver context.
2. The import order initialized the driver API before the runtime API, and the statically linked runtime could not recover.

Resolution. We applied three fixes: (1) recompiled the `.so` with `--cudart shared` so it uses the process-wide `libcudart.so`; (2) added an early `torch.cuda.init()` call in `__init__.py` before importing `cuda_ops`, ensuring the runtime API context is established first; (3) made PyCUDA import lazy (`_ensure_pycuda()`) in `cuda_kernel_ops.py` to defer driver context creation. After these fixes, all CUDA subsystems coexist correctly.

This experience illustrates a class of system challenges that production serving frameworks must handle but that are rarely discussed in algorithm-focused papers: the interaction between multiple CUDA context models within a single process.

5.2 Kernel Design Trade-offs

Our V1 kernel assigns one thread block per (sequence, head) pair. This is the simplest mapping but has several performance implications:

- **Per-token dot product.** Each thread computes the *full* dot product $\mathbf{q} \cdot \mathbf{k}_t$ for its assigned tokens. This works well for small head dimensions (e.g., $D = 64$) but leaves threads idle when $D < 32$ (warp size) during Pass 3, since threads beyond D contribute nothing to value accumulation. The vLLM kernel uses thread-group partitioning where multiple threads cooperate on each dot product.
- **Non-coalesced cache access.** The cache layout $[N_{\text{blocks}}, B, H, D]$ means that consecutive threads in Pass 1 access different context tokens, which may reside in different physical blocks with non-contiguous addresses. A transposed layout (e.g., $[N_{\text{blocks}}, H, D/x, B, x]$ as in vLLM) would enable coalesced reads.
- **Shared memory limit.** Dynamic shared memory of size $(\text{ctx_len} + D + W) \times 4$ bytes limits the maximum context length. For $\text{ctx_len} = 2048$, $D = 128$, $W = 4$,

this is $\sim$8.7 KB—well within the 48 KB SM limit. However, contexts exceeding $\sim$12K tokens would require a multi-block "V2" decomposition.

5.3 Memory Management Overhead

Each kernel invocation currently copies all inputs from host to device and the output back. This is correct but introduces per-call overhead proportional to cache size, which would dominate at large batch sizes. A production system keeps the KV cache GPU-resident; our host-mediated approach is a deliberate simplification for the educational setting.

6 Conclusion and Discussion

We have implemented a functionally complete reimplementation of PagedAttention within MiniTorch, covering the block memory manager, a correctness-validated CUDA kernel, and full transformer integration with prefill/decode generation. The implementation passes 15 comprehensive tests spanning individual components through end-to-end generation.

Key takeaways.

1. The paging abstraction translates cleanly from OS virtual memory to KV cache management: a free list, block tables, and reference-counted physical blocks suffice for the allocator.
2. A three-pass kernel (QK scores $\rightarrow$ stable softmax $\rightarrow$ value accumulation) with warp-level reductions provides a correct and readable implementation of attention over non-contiguous memory, suitable for pedagogical purposes.
3. System-level integration is non-trivial: the CUDA runtime/driver API context conflict required careful import ordering and linking configuration—a class of challenges that production LLM systems must address but that algorithm papers rarely discuss.

Remaining work. For the final report, we plan to:

- Implement and run throughput and memory utilization benchmarks comparing paged vs. contiguous attention.
- Switch the decode path from the Python reference to the CUDA kernel for performance measurement.
- Explore kernel optimizations (coalesced memory layout, thread-group partitioning, online softmax) and quantify their impact.
- Evaluate fragmentation under dynamic workloads with request churn (arrivals and departures).
- Profile the kernel with Nsight Compute to analyze memory bandwidth utilization, occupancy, and warp stall reasons.

7 Limitations

1. **Host-device copies per call.** Each kernel invocation copies all inputs from host memory to GPU and back, rather than keeping the KV cache GPU-resident. This adds latency overhead that would be unacceptable in production.
2. **Single kernel variant.** We implement only the V1 kernel (one thread block per sequence per head). For long contexts (>2K tokens), a V2 multi-block decomposition would be needed to maintain GPU occupancy.
3. **No template specialization.** All head dimensions and block sizes are handled dynamically, forgoing the compiler optimization opportunities that compile-time constants enable.
4. **Non-coalesced memory access.** The $[N_{\text{blocks}}, B, H, D]$ cache layout does not optimize for GPU memory coalescing.
5. **Educational framework overhead.** MiniTorch's surrounding operations (projections, layer norm, embedding) are CPU-based, making end-to-end throughput numbers less meaningful for evaluating kernel performance in isolation.
6. **No benchmarks yet.** Quantitative performance evaluation is deferred to the final report; the current work establishes correctness only.

References

Dao, T. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations (ICLR)*, 2024.

Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Ré, C. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Hugging Face. Text generation inference. https://github.com/huggingface/text-generation-inference, 2023.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, pp. 611–626. ACM, 2023.

NVIDIA. TensorRT-LLM. https://github.com/NVIDIA/TensorRT-LLM, 2024.

Rush, A. MiniTorch: A DIY teaching library for machine learning engineering. https://minitorch.github.io/, 2020.

Silberschatz, A., Galvin, P. B., and Gagne, G. *Operating System Concepts*. Wiley, 10th edition, 2018.

Xiong, R., Yang, Y., He, D., Zheng, K., Zheng, S., Xing, C., Zhang, H., Lan, Y., Wang, L., and Liu, T.-Y. On layer normalization in the transformer architecture. In *International Conference on Machine Learning (ICML)*. PMLR, 2020.

Ye, Z., Chen, L., Lai, R., Lin, W., Zhang, Y., Wang, S., and Chen, T. FlashInfer: Efficient and customizable attention engine for LLM inference serving, 2025.